Symphony of Speeds: Harmonizing Classic McEliece Cryptography with GPU Innovation

Wen Wu, Jiankuo Dong, Zhen Xu, Zhenjiang Dong, Dung Duong, Fu Xiao and Jingqiang Lin

***Abstract*—The Classic McEliece key encapsulation mechanism (KEM), a candidate in the fourth-round post-quantum cryptography (PQC) standardization process by the National Institute of Standards and Technology (NIST), stands out for its conservative design and robust security guarantees. Its deployment is impeded by exceptionally large public and secret keys. Modern GPUs offer abundant parallelism and global memory, making them well suited to such key sizes and to high-throughput cryptographic workloads. However, there has not been a systematic implementation of Classic McEliece on GPU platforms. This paper presents the first high-performance implementation of Classic McEliece on NVIDIA GPUs. Firstly, we present the first GPU-based implementation of Classic McEliece, utilizing a "CPU-GPU" heterogeneous approach and a kernel fusion strategy. We significantly reduce global memory accesses, optimizing memory access patterns. This results in encapsulation and decapsulation performance of 28,628,195 ops/s and 3,051,701 ops/s, respectively, for McEliece348864. Secondly, core operations like Additive Fast Fourier Transforms (AFFT), and Transpose AFFT (TAFFT) are optimized. We introduce the concept of the (T)AFFT stepping chain and propose two universal schemes: Memory Access Stepping Strategy (MASS) and Layer-Fused Memory Access Stepping Strategy (LFMASS), which achieve a speedup of 30.56% and 38.37%, respectively, compared to the native GPU-based McEliece6960119 implementation. Thirdly, extensive experiments on the NVIDIA RTX4090 show significant performance gains, achieving up to 344× higher encapsulation and 125× higher decapsulation compared to the official CPU-based AVX implementation, decisively outperforming existing ARM Cortex-M4 and FPGA implementations.**

***Index Terms*—Post-quantum Cryptography, Classic McEliece, Additive FFT, GPU.**

I. Introduction

In the backdrop of rapid advancements in quantum computing, modern cryptographic algorithms are facing imminent threats. Quantum algorithms, famously proposed by Shor [1] and Grover [2], significantly reduce the security stature of prevalent asymmetric systems like Rivest-Shamir-Adleman (RSA) [3] and Elliptic Curve Cryptography (ECC) [4]. Consequently, apprehensions arise concerning the future of cryptography and the confidentiality of communications across the Internet and allied networks. To uphold secure communication channels, Post-Quantum Cryptography (PQC) endeavors to withstand attacks orchestrated by quantum computers. On July 5, 2022, the National Institute of Standards and Technology (NIST) of the United States released its inaugural set of four algorithms, comprising a key establishment algorithm named CRYSTALS-Kyber, and three digital signature algorithms known as CRYSTALS-Dilithium, FALCON, and SPHINCS+ [5]. Concurrently, an extension of the NIST PQC standardization initiative (4th round) outlines key establishment algorithms such as BIKE [6], HQC [7], and Classic McEliece [8], which are all rooted in error-correcting codes.

Classic McEliece is a code-based cryptosystem known for its strong post-quantum security guarantees and decades of rigorous cryptanalysis. Its resilience makes it a compelling choice for high-assurance applications such as critical infrastructure, long-term data protection, and secure government or military communications. Despite its security strengths, Classic McEliece faces practical deployment challenges due to its large public key size and relatively high computational cost. These limitations are particularly pronounced in key encapsulation mechanisms (KEMs) deployed within performance-sensitive protocols, such as post-quantum TLS, VPNs, and secure IoT frameworks, where fast and scalable key exchange is essential. In environments like large-scale server clusters or resource-constrained edge devices, systems must process a high volume of encapsulation and decapsulation operations with minimal latency. Inefficient implementations can therefore become significant bottlenecks. Enhancing the performance of Classic McEliece KEMs is thus crucial to ensuring the scalability, practicality, and responsiveness of high-security systems.

A. Related works

The inception of code-based cryptosystems dates back to McEliece's pioneering work in 1978 [9]. Classic McEliece is a modified version of this original scheme, described in the fourth round competition [10] by NIST. Notably, Classic McEliece's submission to the NIST competition included four distinct software implementations coded in the C language: ref, vec, sse, and avx [11]. Among these, the ref implementation serves primarily as an illustrative tool elucidating the scheme's operational mechanics, prioritizing clarity over optimization. The vec implementation harnesses vectorization techniques tailored to 64-bit integers, while the sse and avx variants leverage specialized instructions specific to Intel/AMD architectures, specifically 128-bit and 256-bit vector instructions, respectively. NIST held the 5th NIST PQC Standardization Conference from April 10-12, 2024. The aim of this international conference was to discuss various aspects of algorithms, including those selected and under evaluation, and to gather feedback for informing decisions on standardization.

W. Wu, J. Dong, Z. Xu, Z. Dong and F. Xiao are with School of Computer Science, Nanjing University of Posts and Telecommunications. H. Duong is with School of Computing and Information Technology, University of Wollongong. J. Lin is with School of Cyber Security, University of Science and Technology of China.
Corresponding author: ***Jiankuo Dong****, djiankuo@njupt.edu.cn.*

Numerous efforts have been made to augment the computational efficiency of Classic McEliece algorithms through the utilization of various accelerators. Chen [12] introduced a groundbreaking constant-time implementation of Classic McEliece specifically tailored for ARM Cortex-M4 architecture. Building upon the vec implementation as a foundation, they ingeniously leveraged flash memory to accommodate oversized public keys, thereby striving to curtail the frequency of memory accesses. Their concerted efforts resulted in significantly accelerated encapsulation and decapsulation times compared to the native vec implementation. Sim et al. [13] harnessed the capabilities of vector registers and vector instructions specific to the ARMv8 processor to implement sophisticated parallel computing techniques. Importantly, they focused on optimizing the crucial multiplication operations, which significantly contribute to the overall execution time of the Classic McEliece algorithm. Chen et al. [14] and Lin et al. [15] implemented Classic McEliece on the FPGA platform. Bernstein et al. [16] introduced McBits, a groundbreaking constant-time, bitsliced implementation of a cryptosystem based on the Niederreiter variant [17] of the McEliece cryptosystem. Building upon this foundation, Chou [18] proposed bitsliced implementation in 2017, interweaving elements such as Additive Fast Fourier Transforms (AFFT) [19], Transpose AFFT (TAFFT), the Berlekamp-Massey (BM) algorithm, and Beněs networks [20] to achieve enhanced efficiency.

In cryptographic engineering, leveraging Graphics Processing Unit (GPU) general-purpose computing to accelerate cryptographic algorithms has emerged as a compelling avenue, offering notable computational advantages. GPUs stand out as exceptionally efficient platforms for executing cryptographic computations, harnessing parallelism to maximize performance. Several seminal works have yielded valuable insights into the parallel acceleration of cryptographic algorithms on GPU platforms. Gao et al. [21] made notable strides by harnessing GPU acceleration for the ECC algorithm, achieving throughput performances exceeding ten million operations per second, which represents a significant leap compared to CPU-based implementations. Ji et al. [22] presented a high-performance implementation of Kyber on NVIDIA GPUs. Narisada et al. [23] pursued a lightweight approach by implementing the Dumer algorithm for Information Set Decoding (ISD) on GPU platforms, thereby solving a 1161-dimensional Syndrome Decoding Problem (SDP) in the Goppa-McEliece cryptosystem. Furthermore, their subsequent work [24] involved implementing the improved May Maurer Thomae (MMT) algorithm in a GPU environment, culminating in the establishment of a new performance record for the McEliece-1409 instance.

B. Our Contribution

Based on the powerful parallel computing capabilities of GPU platforms, some existing research has explored the implementation of PQC algorithms (such as Kyber, Dilithium, and others) on GPUs. However, to the best of our knowledge, no systematic implementation of Classic McEliece on GPUs exists in the published related work. Thus, considerable research opportunities remain for enhancing the performance of GPU-based implementations of Classic McEliece. The contributions of this paper are summarized as follows:

- Firstly, we are the first to implement the Classic McEliece on GPU platforms using a "CPU-GPU" heterogeneous computing approach. We design a novel and complete high-performance implementation scheme of Classic McEliece encapsulation and decapsulation algorithms by adopting a kernel fusion strategy, which greatly reduces the number of global memory accesses during computation. Based on the proposed general parallel architecture, we primarily optimize it in terms of memory access. Ultimately, the optimal encapsulation performance of McEliece348864 reaches 28,628,195 ops/s, and the decapsulation performance reaches 3,051,701 ops/s.
- Secondly, we specifically optimize the core operations of Classic McEliece decapsulation, including AFFT, TAFFT. Addressing the calculation bottleneck of (T)AFFT, we introduce the concept of (T)AFFT stepping chain based on its characteristics. Additionally, we propose two schemes that are universal on other platforms: Memory Access Stepping Strategy (MASS) and Layer-Fused Memory Access Stepping Strategy (LFMASS), which achieve a speedup of 30.56% and 38.37%, respectively, compared to the native GPU-based McEliece6960119 implementation. Furthermore, we optimize AFFT using the layer fusion scheme proposed in LFMASS and improve memory usage efficiency in the data conversion operations.
- Finally, we conduct comprehensive performance experiments with different parallel dimensions. We optimize Classic McEliece's five parameter sets on the NVIDIA RTX4090, performing extensive experiments to find the optimal parallel parameters and achieve peak performance. Additionally, the performance improvement rate of the proposed optimization methods across different parameter sets and the scalability of different NVIDAI GPUs are analyzed. Our GPU-based implementation increases encapsulation performance by up to 344× and decapsulation performance by up to 125× compared to the official CPU-based AVX implementation. Additionally, our implementation outperforms existing ARM Cortex-M4 and FPGA implementations and is compared with GPU implementations of related PQC algorithms.

II. Preliminaries

This section introduces the preliminaries. We briefly introduce the GPU architecture and the Classic McEliece.

A. GPU Architecture

The GPU architecture employs scalable multi-threaded Streaming Multiprocessors (SMs) [25] that execute in units of 32 threads, known as warps, under the Single-Instruction Multiple-Thread (SIMT) [26] model. This model enables simultaneous execution of identical instructions across threads within a warp, though branch divergence requires sequential execution of divergent paths. To optimize performance, minimizing warp divergence is crucial. Each SM independently

manages its execution context, including registers, data caches, and shared memory, accommodating multiple warps based on kernel requirements and available resources. CUDA [27], introduced by NVIDIA in 2006, provides a parallel computing framework where the GPU acts as a coprocessor to the CPU, with separate memory spaces. The CUDA programming model facilitates kernel execution on the GPU and management of memory and data transfers through CUDA runtime APIs.

B. Classic McEliece

The first code-based cryptosystem was introduced by McEliece in 1978 [9], utilizing a randomly selected irreducible binary Goppa code. Subsequently, in 1986, Niederreiter [17] introduced a dual variant of the McEliece cryptosystem, employing a parity-check matrix for encryption instead of a generator matrix. The Classic McEliece KEM is constructed upon the Niederreiter framework and offers an IND-CCA2-secure [28] key encapsulation mechanism (KEM). This is achieved through a standard transformation of an IND-CPA-secure Niederreiter-style public-key encryption scheme (PKE).

TABLE I
CLASSIC MCELIECE PARAMETER SETS

	Public key	Private key	Ciphertext	Session key
McEliece348864	261,120	6,492	96	32
McEliece460896	524,160	13,608	156	32
McEliece6688128	1,044,992	13,932	208	32
McEliece6960119	1,047,319	13,948	194	32
McEliece8192128	1,357,824	14,120	208	32

This enduring security pedigree renders Classic McEliece a stalwart cryptosystem, instilling confidence among practitioners. However, the confidence comes at a price: the public keys are quite large. Notably, Classic McEliece offers a variety of parameter sets, ranging from level-1, characterized by public key sizes around 256 KB, to level-3 with sizes around 512 KB, and extending to level-5, with sizes reaching approximately 1 MB. Furthermore, certain level-5 parameter sets feature even larger sizes, which approach 1.3 MB. Table I shows the sizes of public keys, private keys, ciphertexts, and session keys for each selected parameter set. All sizes are expressed in bytes.Next, we describe the IND-CPA-secure PKE.

Encryption. Using the public key $\mathbf{T}$, the sender reconstructs the Goppa parity check matrix $\widehat{\mathbf{H}}$ in its systematic form $[\mathrm{I}_{mt}|\mathbf{T}]$. Next, the sender selects an error vector $\mathbf{e} \in \mathbb{F}_2^n$ with a Hamming weight of $w_H(\mathbf{e}) = t$ to serve as the plaintext. The sender then calculates the syndrome $\mathbf{s} = [\mathrm{I}_{mt}|\mathbf{T}]\mathbf{e}^\top$, which constitutes the ciphertext.

Algorithm 1 Encryption for the PKE

Input: Plaintext $\mathbf{e} \in \mathbb{F}_2^n$, public key $\mathbf{T}$;
Output: Ciphertext s;
Return $s = \widehat{\mathbf{H}}\mathbf{e}^\top = [\mathbf{I}_{mt}|\mathbf{T}]\mathbf{e}^\top$;

Decryption. The decryption process for Classic McEliece involves the syndrome decoding of binary Goppa codes. This includes computing the syndrome polynomial, finding the error locator polynomial, and evaluating the error locator polynomial at points in $\mathbb{F}_{2^m}$. According to [16], [29], official implementations of Classic McEliece utilize the constant-time Berlekamp-Massey (BM) algorithm for decoding. Leveraging a technique described by Sendrier in [30], it is possible to correct t errors by constructing a parity-check matrix $\mathbf{H}^{(2)}$ of size $2t \times n$ over $\mathbb{F}_{2^m}$. This matrix is then transformed into a binary matrix $\mathbf{H}'^{(2)}$ of size $2mt \times n$ by replacing each entry with an m-bit column. The double-size syndrome $\mathbf{s}^{(2)}$ is computed as $\mathbf{H}'^{(2)}[\mathbf{s}|0]^\top$, where $n - mt$ zeros are appended to the syndrome $\mathbf{s}$. The constant-time BM algorithm is then employed to determine the error locator polynomial $\sigma(x) \in \mathbb{F}_{2^m}[x]$ from $\mathbf{s}^{(2)}$ and to evaluate $\sigma(x)$ over all elements in $\mathbb{F}_{2^m}$. This polynomial evaluation can be efficiently conducted using the additive FFT. Finally, the partial secret key $(\alpha_1, \alpha_2, \ldots, \alpha_n)$ is read, and each bit e_i of the error vector is set to 1 if $\sigma(\alpha_i) = 0$ and 0 otherwise.

Algorithm 2 Decryption for the PKE

Input: Ciphertext s, secret key $((g(x), (\alpha_1, \alpha_2, ..., \alpha_n))$;
Output: Plaintext $\mathbf{e}$;
1: Generate a $2t \times n$ expanded parity check matrix $\mathbf{H}^{(2)}$;
2: Transform matrix $\mathbf{H}^{(2)}$ into a binary matrix $\mathbf{H}'^{(2)}$ of dimensions $2mt \times n$, encoding each element as a column of m bits;
3: Calculate the double-size syndrome $s^{(2)} = \mathbf{H}'^{(2)}[\mathbf{s}|\mathbf{0}]^\top$;
4: Apply the Berlekamp-Massey algorithm to ascertain the polynomial $\sigma(x)$, which identifies the locations of errors;
5: Evaluate $\sigma(x)$ at $(\alpha_1, \alpha_2, ..., \alpha_n)$ and recover plaintext $\mathbf{e}$;
6: Return the plaintext $\mathbf{e}$;

III. OPTIMIZED IMPLEMENTATION ON GPU

The detailed optimization strategies of Classic McEliece on GPU are discussed in this section. Firstly, we show the overall framework of Classic McEliece. Then, the design for encapsulation is given. Based on the vec implementation, we introduce the optimization methods of TAFFT, AFFT, and data conversion one by one.

A. Overall Framework of Classic McEliece Implementation

In GPU-based algorithm design, implementation strategies are generally classified into coarse-grained and fine-grained parallelism. The coarse-grained model assigns each thread the responsibility of independently executing a full encapsulation or decapsulation process. This approach reduces the overhead of inter-thread communication and simplifies synchronization, which is beneficial for maximizing throughput. In contrast, fine-grained implementations distribute the computation of a single operation across multiple threads, requiring coordination and potentially increasing latency. To fully leverage the parallel processing capabilities of the GPU and achieve high throughput, we employ a kernel fusion strategy and adopt the coarse-grained strategy, wherein each thread handles one complete encapsulation or decapsulation operation independently. This also greatly reduces the number of global memory accesses during computation and improves the throughput.

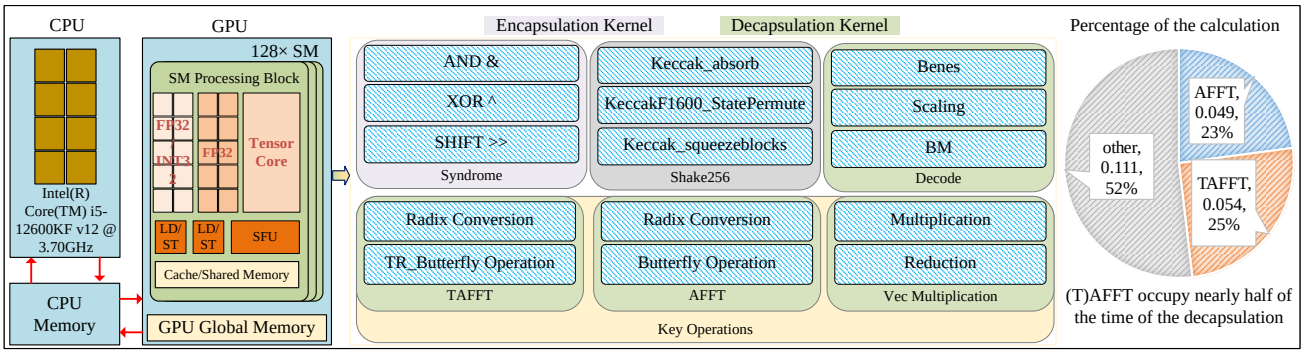


Fig. 1. Overall Framework of GPU-based Classic McEliece

According to Section II-B, the implementation of Classic McEliece on the GPU can use either the ref or vec versions. For the encapsulation, a comparative analysis of these two implementations reveals that they are essentially the same, and the final choice of implementation will be detailed in the following section. For the decapsulation, we implement the ref and vec versions of McEliece8192128 on the RTX4090, shown in Table II. The peak throughput of the native GPU-based ref implementation is 57,192 ops/s, with a significant latency of 1,145 ms. In contrast, the performance of the native GPU-based vec implementation is 694,957 ops/s, with a latency of approximately 23 ms, far exceeding that of the GPU-based ref implementation. Therefore, we opt for the native GPU-based vec implementation as the baseline and further optimize it.

TABLE II
PERFORMANCE COMPARISON OF NATIVE REF AND VEC FOR MCELIECE8192128 DEC.

Threads	Ref		Vec	
	Throughput (ops/s)	Latency (ms)	Throughput (ops/s)	Latency (ms)
128×64	18,079	453.129	642,349	12.753
128×128	29,529	554.839	**694,957**	23.576
128×256	42,355	773.656	613,853	53.381
128×384	52,126	942.948	528,248	93.047
128×512	**57,192**	1,145.901	489,539	133.873

Figure 1 shows the overall composition framework of GPU-based Classic McEliece. Through testing, we find that in the CPU implementation, the computation of (T)AFFT nearly accounts for half of the total decapsulation. Therefore, we first focus on optimizing (T)AFFT operation.

B. Design for Encapsulation on GPU

Chen et al. [12] implemented the encapsulation algorithm of Classic McEliece on the STM32F4-Discovery development board. However, the processor of this development board is an ARM Cortex-M4 with only 192KB of SRAM, which evidently cannot fully accommodate the public key due to its limited memory capacity. To address this limitation, they utilized the 1MB flash memory of the board to store the public key and employed a block-by-block computation approach for the encapsulation process. Their implementation followed the strategy of the vec version.

Similarly, the GPU platform utilized in our study also faces similar challenges. Due to the exclusive register allocation per thread in GPUs, the register count is significantly higher compared to that in CPUs. However, this allocation remains insufficient for accommodating the public key. For instance, on the GTX Titan, each SM possesses 65,536 32-bit registers, totaling up to 256 KB of register space. When register resources within a thread are insufficient, some variables are "spilled" into local memory. However, GPU platforms offer extensive global memory capacities. For instance, the NVIDIA GTX Titan boasts 6GB of global memory, while the NVIDIA GeForce RTX4090 [31] provides 24GB of global memory. Taking McEliece8192128 as an example, during the testing of the GPU implementation, it was observed that attempting to store the public key in registers resulted in excessive resource consumption, rendering the program unable to execute correctly, let alone achieve any performance breakthroughs. Fortunately, the oversized public key can be accommodated in global memory. Although access to global memory is slower, the program operates correctly under this configuration.

We integrate the characteristics of GPU platforms to design a "CPU-GPU" heterogeneous computing approach to implement the Classic McEliece encapsulation algorithm. Initially, the key generation algorithm is executed on the CPU, successfully generating public and private keys. Subsequently, the public keys are copied to the GPU using $CudaMemcpy$, with the public keys stored in global memory. Finally, each thread executes an encapsulation operation to obtain ciphertext s, which is then transferred back to the CPU by the GPU. In Section II-B, the principles of the encapsulation algorithm have been explained, with its specific implementation essentially comprising vector multiplication. Due to the significant number of zero elements present in the identity matrix $\mathbf{I}$, many redundant operations are introduced during this process, which negatively affect the performance of encapsulation. In contrast, the computations involving the public key matrix $\mathbf{T}$ are both essential and unavoidable. Building upon this observation, we implement and optimize the encapsulation algorithm on NVIDIA GPUs, as depicted in Fig. 2, where each encapsulation calculations within a thread are performed.

In the encapsulation, the ref implementation utilizes the $uint8\_t$ type, while the vec implementation employs the $uint64\_t$ type. Our analysis indicates that, except for the

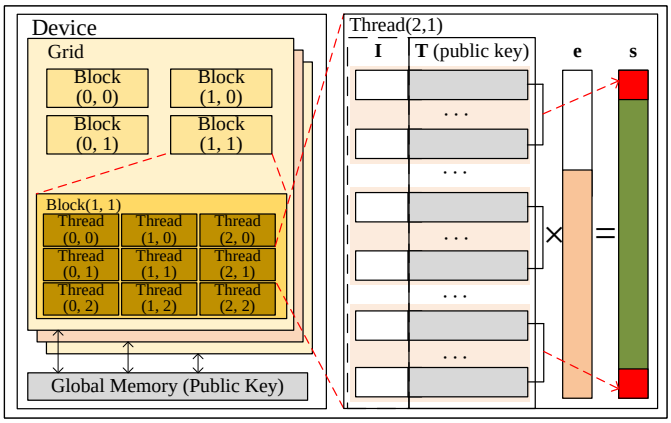


Fig. 2. GPU-based Encapsulation

McEliece8192128 parameter set, the use of $uint64\_t$ in the other four parameter sets leads to unaligned memory access. On CPU platforms, unaligned access with $uint64\_t$ is generally permissible and does not cause execution failures. However, in the CUDA environment on GPU platforms, data types larger than 4 bytes, such as $uint64\_t$, must be aligned to memory addresses that are multiples of their size. If unaligned memory is forcibly cast to a $uint64\_t*$ and accessed, it may result in runtime errors. Therefore, to ensure correct memory alignment during encapsulation on the GPU, we adopt a $uint32\_t$-based byte-wise access strategy. Specifically, for the GPU-based McEliece6960119 encapsulation, we opt for computations using $uint8\_t$, as both $uint32\_t$ and $uint64\_t$ lead to unaligned memory access issues in this case.

C. Transpose Additive FFT

In the vec implementation of the Classic McEliece decapsulation algorithm, core computations involve additive FFT, transpose additive FFT, etc. To facilitate the description, we first introduce the optimized implementation of the transpose additive FFT, influenced by the computational structure of its core butterfly operations. The butterfly operation process of the transpose additive FFT in the native vec implementation is depicted in Algorithm 3. The core butterfly unit (lines 6-12) consists of a set of vector addition (lines 6-8) and multiply-add (lines 9-12) operations. To execute the multiply-add operation, we initially develop a comprehensive function called $vec\_mulAdd()$. This function allows the computation, which previously required two steps, to be executed in a single function call. Additionally, it eliminates the necessity for the temporary variable tmp during the multiply-add operation.

Figure 3 visually illustrates the butterfly operation process in the transpose additive FFT of an N-dimensional vector, with N being 64 in McEliece348864 and 128 in other parameter sets. Each unit shaped like an "X" represents a pair of butterfly wings, in which "↗" represents a multiply-add operation and "↘" represents an addition operation. This structure essentially corresponds to a single butterfly unit. As shown, the butterfly operation consists of $\log N - 1$ layers (counting from 0), each involving operations on N groups of vectors. In the native vec implementation, the results of layer

Algorithm 3 Butterfly operation in the transpose additive FFT

Input: Array in$[\cdot][\cdot]$, constants array consts$[\cdot]$;
Output: Modified array in$[\cdot][\cdot]$;

1: **for** $i = \log N - 1$ **down to** 0 **do**
2: $s = 1 \ll i$;
3: consts_ptr$-= s$;
4: **for** $j = 0$ **to** N **by** $2 \cdot s$ **do**
5: **for** $k = j$ **to** $j + s - 1$ **do**
6: **for** $b = 0$ **to** GFBITS $- 1$ **do**
7: in$[k][b] \leftarrow$ in$[k][b] \oplus$ in$[k + s][b]$;
8: **end for**
9: vec_mul(tmp, in$[k]$, consts[consts_ptr $+ (k - j)$]);
10: **for** $b = 0$ **to** GFBITS $- 1$ **do**
11: in$[k + s][b] \leftarrow$ in$[k + s][b] \oplus$ tmp$[b]$;
12: **end for**
13: **end for**
14: **end for**
15: **end for**

0 ($i = \log N - 1$) are computed first, followed by sequential computation of layers 1 to $\log N - 1$. Each layer contains $N/2$ groups of butterfly units, each with a different data span. The data span in layer j is given by $N/2^{j+1}$. For example, in layer 0 ($i = 6$), the data span for any butterfly unit is 64; in layer 3 ($i = 3$), the data span is 8; and in layer 6 ($i = 0$), the data span is only 1. This indicates that as the layer number increases, the data density in butterfly unit computations increases correspondingly.

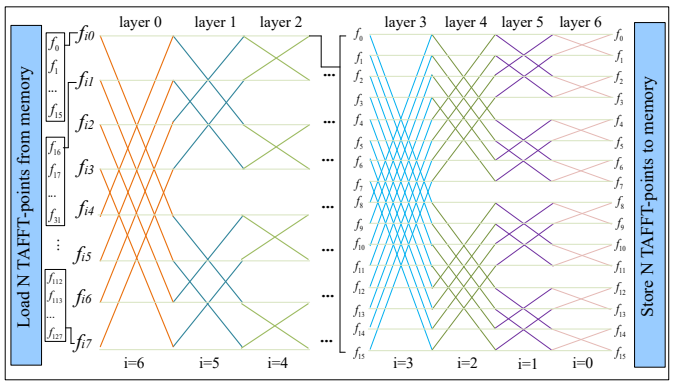


Fig. 3. Calculation Steps of Transpose Additive FFT

The FFT utilizes a recursive divide-and-conquer approach to break down the original problem into smaller, similar subproblems. These subproblems are subsequently solved, and their solutions are combined to solve the original problem. Fig. 3 shows that each layer contains $2^{\log N - i - 1}$ independent groups of butterfly operations. Based on this characteristic, we introduce a generalized memory access stepping strategy (MASS).

*1) **Memory Access Stepping Strategy**:* To facilitate the description, we first introduce the concept of a memory access stepping chain. In the computation process of the transpose additive FFT, we view the entire calculation as a stepping chain. The stepping chain begins with the initial $N/2$ groups of butterfly operations. As the chain progresses, it decomposes

into multiple groups of independent butterfly operations, which are executed sequentially. Each group of butterfly operations is a node in the stepping chain. By executing these independent nodes sequentially, the process of butterfly operation in the transpose additive FFT is ultimately completed.

As depicted in Fig. 4, the stepping chain at the j-th layer comprises 2^j nodes, each node contains $2^{(\log N-j-1)}$ butterfly units, meaning each node contains multiple butterfly units. As the layer number increases, the memory access required for each butterfly operation becomes progressively independent and halves, which is the origin of the term "stepping". Within this stepping chain, each node is assigned a unique identifier [(1), (2), (3), ...] representing the execution sequence of butterfly units. The initial node (1) corresponds to the 64 butterfly units in layer 0, followed by the computation of the subsequent nodes, (2) and (3), each with 32 butterfly units, and so forth.

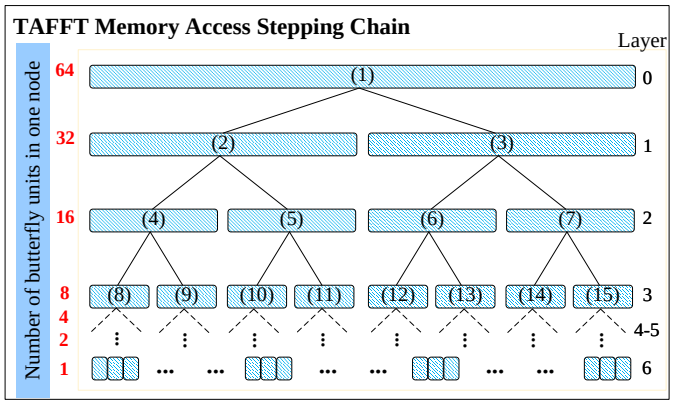


Fig. 4. TAFFT Memory Access Stepping Chain

The native strategy entails traversing all N TAFFT coefficients at each layer, which limits register availability for manipulating these coefficients. This implies that if the native strategy is employed, the computation of each layer will uniformly access the N-dimensional vector, resulting in $N(\log N - 1)$ memory accesses. Therefore, we propose an innovative general memory access stepping strategy to optimize the memory access pattern in the transpose additive FFT computation process. By employing a more compact memory access approach, MASS increases cache hit rates and enhances computational speed.

Figure 5 illustrates the MASS for TAFFT. Due to space constraints, only a portion of the MASS is depicted. Beginning at the initial node, the traversal proceeds vertically down the left subchain until a terminal node is reached. The process then backtracks to the previous node to traverse the right subchain, continuing in this manner until all accessible nodes have been traversed. In other words, the MASS first completes all butterfly operations of length N in layer 0 for $[f_0, f_1, ..., f_{N-1}]$. After traversing the initial node, it moves to the left subsequent node and positions at the initial node of the left subchain, executing the butterfly operations of length $N/2$ in layer 1 for $[f_0, f_1, ..., f_{N/2-1}]$. It continues to traverse the left subchain vertically until completing the final layer's butterfly operations of length 2 for $[f_0, f_1]$, then stops traversing the left subchain. The search then backtracks to the previous node and repositions at the preceding node. Next, it moves to the right subsequent node of the right subchain to calculate the butterfly operations of length 2 for $[f_2, f_3]$. By continuously backtracking until the entire stepping chain is traversed, the MASS is executed.

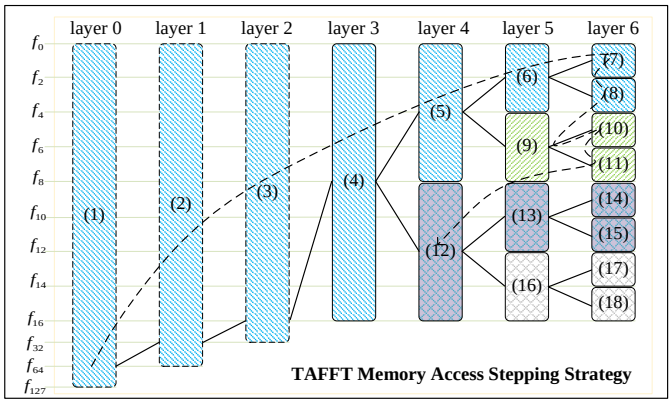


Fig. 5. TAFFT Memory Access Stepping Strategy

In Fig. 5, each node in the jth layer contains $2^{\log N-j-1}$ butterfly units. The nodes in layer $\log N-1$ are terminal nodes, with each terminal node corresponding to a single butterfly unit. Upon reaching node $\log N-1$ during the MASS, further downward traversal becomes impossible. Consequently, backtracking to node (6) becomes necessary, followed by traversing its right node, (8). Once nodes (7) and (8) have been computed, it indicates the derivation of the final $[f_0, f_1, f_2, f_3]$. As a result, these four sets of vectors no longer necessitate reloading and storage. Similarly, upon the completion of computation for nodes (10) and (11), the set $[f_0, f_1, f_2, f_3, f_4, f_5, f_6, f_7]$ similarly does not require further memory access. In essence, as the MASS advances, whenever all terminal nodes of any subchain within this stepping chain are executed, the corresponding sets of vectors complete their final computations, eliminating the need for reloading and storage from memory.

In the native implementation of stepping chain, each layer loads and stores N sets of coefficients, computing from the starting position to the end in sequence, with each coefficient length being N for each calculation. The memory access contiguity between layers is relatively low. The benefit of utilizing MASS lies in enhancing memory contiguity, increasing cache hit rate, and improving computational speed. When employing MASS, the data span between accessed coefficients decreases with the increasing length of the stepping chain. The coefficient span is N when computing layer $\log N - 7$, and it decreases to $N/2$ for layer $\log N - 6$. Similarly, in layer $\log N - 5$, the data span becomes $N/4$, continuing this pattern. Eventually, when computing the terminal nodes (layer $\log N - 1$), the coefficient span narrows to just 2. Compared to the native approach, our MASS proves to be more efficient in terms of cache utilization. This is because the required data for adjacent computations is now "closer" together, allowing for the possibility of keeping this data in registers without the need for frequent loading and storing from the global memory. Experimental results show that while employing the

MASS enhances performance, the code is very verbose, which increases the compilation burden.

*2) **Layer-Fused Memory Access Stepping Strategy**:* As previously introduced, the butterfly operation in the transpose additive FFT involves a total of $\log N$ layers. Applying MASS to each layer ultimately increases the stepping length, resulting in $N/2$ terminal nodes. However, the butterfly operation layers cannot be directly reduced. This requires multiple backtracking steps, which increases the algorithm's code complexity. Given the verbosity and compilation burden of the MASS implementation, based on the MASS, we propose a layer-fused memory access stepping strategy (LFMASS). By reducing the original $\log N$ layers through a layer fusion scheme, the problem is then addressed using MASS.

We describe the final layer fusion scheme we adopt as "1|2|4". As illustrated in Fig. 6, this scheme involves redividing the original $\log N$-layer butterfly operation into three layers. Specifically, "1" represents the new first layer, aligning with the original layer 0; "2" corresponds to the new second layer, encompassing the original layers 1 and 2; and "4" corresponds to the new third layer, encompassing the original layers 3 through 6. There are indeed various layer fusion schemes available, where the original $\log N$-layer butterfly operation can be fused into configurations such as "3|4" or "1|1|5". These different fusion schemes have varying impacts on computational performance. The reason for the detailed analysis of the "1|2|4" scheme in this paper is that through comparative analysis of different layer fusion schemes, the superiority of this scheme has been established. Section IV-A discusses the performance impact of different layer fusion.

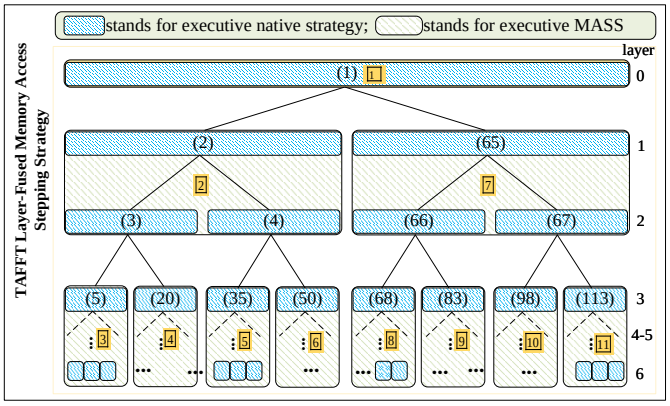


Fig. 6. Layer-Fused Memory Access Stepping Strategy

In the "1|2|4" layer fusion scheme, the second layer consists of two nodes, each containing a subchain with a length of 2, while the third layer consists of eight nodes, each containing a subchain with a length of 4. MASS can still be applied to traverse each node. First, we compute the butterfly operations for the N-coefficients in the first layer. Next, the left subchain is processed, followed by its four nodes in sequence. After completing these computations, backtracking computes the initial node's right subchain, and finally, the remaining four nodes are processed. In the stepping chain represented by the layer fusion scheme, each terminal node includes a four-layer stepping chain. There are two possible strategies for the computations within each terminal node: native strategy and MASS. It is important to note that MASS would lead to similar issues like code redundancy and increased compilation burden. Therefore, the native strategy is used to compute each new terminal node.

```
#define TR_BUTTERFLY(R,S,T,U,V)                          \
    consts_ptr = R;                                      \
    for (i = S; i >= T; i--) {                           \
        s = 1 << i;   consts_ptr -= s;                   \
        for (j = U; j < V; j += 2*s)                     \
            for (k = j; k < j+s; k++) {                  \
                for (b = 0; b < GFBITS; b++)             \
                    in[k][b] ^= in[k+s][b];              \
                vec_mulAdd(in[k+s], in[k],               \
                    consts[consts_ptr + (k-j)]);         \
            }                                            \
    }                                                    \
```

LFMASS combines the advantages of both strategies. Utilizing the MASS concept reduces the data span for butterfly operations at each layer and increases cache hit likelihood during computation. Additionally, it leverages the benefits of native strategy to reduce code redundancy and improve compilation efficiency. To facilitate the implementation of the LFMASS, we define a macro named $TR\_BUTTERFLY(R, S, T, U, V)$, where R represents a constant corresponding to each layer, S and T represent the layer numbers, and U and V denote the range of coefficients involved in that layer.

Macros compress lengthy code snippets into concise expressions, enabling code replacement and expansion during compilation without runtime function calls. This enhances program performance, particularly in situations requiring frequent invocations. Our LFMASS implementation for the transpose additive FFT butterfly operation in McEliece8192128 is structured as illustrated in Table III. Line 1 corresponds to all butterfly operations in layer 0 following layer fusion. Line 2 corresponds to the left child of the initial node in layer 1 post-layer fusion. Lines 3 and 4 correspond to the four terminal nodes of the left subchain in layer 2 post-layer fusion. Line 5 represents the backtracking computation for the right child of the initial node after completing the MASS traversal of the left subchain. Lines 6 and 7 correspond to the four terminal nodes of the right subchain in layer 2 post-layer fusion.

TABLE III
LFMASS IMPLEMENTATION FOR TAFFT

1.TR_BUTTERFLY(128,6,6,0,128);	
2.TR_BUTTERFLY(64,5,4,0,64);	
3.TR_BUTTERFLY(16,3,0,0,16);	TR_BUTTERFLY(16,3,0,16,32);
4.TR_BUTTERFLY(16,3,0,32,48);	TR_BUTTERFLY(16,3,0,48,64);
5.TR_BUTTERFLY(64,5,4,64,128);	
6.TR_BUTTERFLY(16,3,0,64,80);	TR_BUTTERFLY(16,3,0,80,96);
7.TR_BUTTERFLY(16,3,0,96,112);	TR_BUTTERFLY(16,3,0,112,128);

D. Additive FFT

The additive FFT procedure comprises two steps: the radix conversion and twisting step, which transforms the input polynomial $\sigma(x)$ into several constant polynomials with a single coefficient of 1, and the reduction step, which iteratively evaluates these polynomials at the input points.

In the implementation of the additive FFT, a butterfly operation is also involved. However, unlike the butterfly operation in the transpose additive FFT, the AFFT butterfly distances increase layer by layer. The distances of butterflies at each layer are $[2, 4, 8, 16, 32, 64]$, resembling the TAFFT stepping chain's backtracking process. For clarity, Fig. 7 illustrates our method, omitting the complete original butterfly operations of the AFFT. During the backtracking process, the upper AFFT layers depend on the lower ones. For instance, in AFFT, node (9) depends on the values of the underlying node (1) and the underlying node (2), and ultimately node (13) depends on the values of nodes (9) and (10). If we get the value of (9) by calculating the underlying nodes (1) and (2), its memory cannot be freed until node (10) is calculated. The computation of node (13) needs its left and right child nodes. Likewise, if node (14) is not calculated, node (13) cannot be released. In this case, MASS cannot be used because the preceding node's value depends on its left and right child nodes.

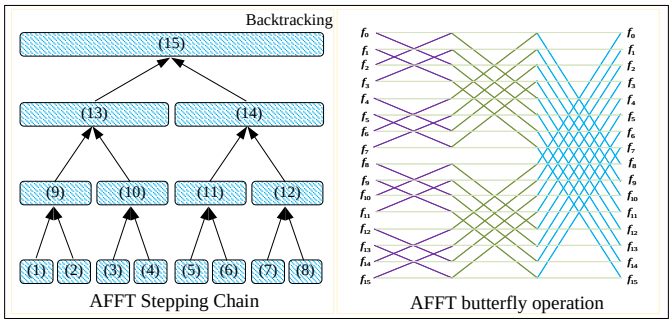


Fig. 7. Additive FFT

However, our proposed LFMASS can also be applied to the additive FFT. Layer fusion divides a large stepping chain into several smaller subchains, computed using the native strategy. In implementation, it is not necessary to always merge to the initial node, as this is infeasible. Instead, we first compute the subchain [(1), (2), (9)] using the native strategy, then compute the subchain [(3), (4), (10)], followed by the subchain [(9), (10), (13)]. Using the same method, we compute node (14), and finally, nodes (13) and (14) to compute node (15). The advantage of using the layer fusion strategy is that by decomposing a large stepping chain into smaller subchains and employing the native method within each subchain, the strategy aligns with the characteristics of butterfly operations at each layer. This approach reduces the data span required for coefficient calculations across layers, increases cache hit probability, and enhances computational speed.

In the implementation process, we convert the 6 layers of AFFT butterfly operations into 3 layers using the layer fusion scheme, specifically described as "4|1|1". The "4" represents the fusion of layers 0-3 into four terminal nodes, the middle "1" represents layer 4, and the third "1" represents layer 5 (the initial node of the AFFT stepping chain). Similar to the transpose additive FFT, there are various layer fusion schemes, such as "3|2|1" or "4|2." This paper selects "4|1|1" because it offers the better performance. A comparative performance analysis of different layer fusion schemes will be presented in Section IV-A.

E. Memory Optimization

Field arithmetic operations within $\mathbb{F}_{2^m}$ (where $m \in \{12, 13\}$), particularly field multiplications, are pivotal elements within the decoding algorithm. As specified, $\mathbb{F}_{2^{12}}$ is defined as $\mathbb{F}_2[x]/(x^{12}+x^3+1)$, and $\mathbb{F}_{2^{13}}$ as $\mathbb{F}_2[x]/(x^{13}+x^4+x^3+x+1)$. In the vec implementation of Classic McEliece, the representation of field elements takes two distinct forms. The first is the bitsliced representation, primarily employed for additive FFT and transpose additive FFT. The second form is the "radix-16" representation, exclusively utilized within the Berlekamp-Massey (BM) algorithm.

Therefore, in the vec implementation, data conversion between these two representations is necessary. We optimize the function $get\_coefs(gf * out, vec * in)$ to convert data from bitsliced to "radix-16" representation. As depicted in Fig. 8, the native implementation involves defining a two-dimensional array $mask[4][2]$ and initializing it using the $vec\_set1\_16b()$ function 8 times. In actual implementation, each element of this array is used only once with no mutual dependency. Therefore, we reduce this two-dimensional array to a one-dimensional array $mask[2]$, continuously updating its values through assignment during computation. By reusing registers, minimizing function invocations, and reducing memory usage, we expect to improve the speed of data conversion.

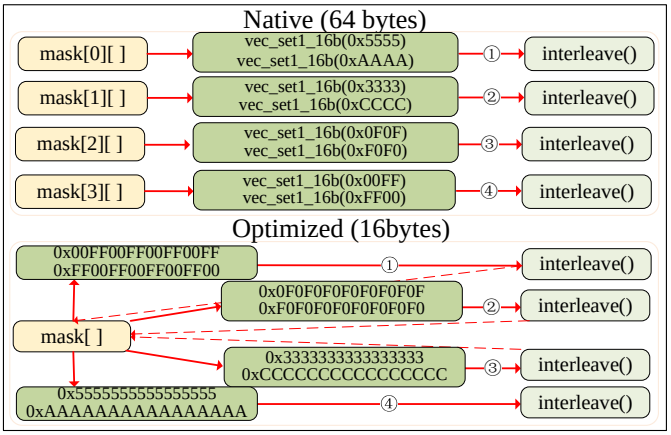


Fig. 8. Memory optimization

Additionally, in the GPU-based decapsulation implementation of Classic McEliece, we efficiently use the global memory, constant memory, and registers to ensure smooth program operation and improved decapsulation performance. Experimental results show that storing the private key in global memory is more efficient for decapsulation. In the vec implementation, several sets of constant coefficients are required for the transpose additive FFT and additive FFT processes. Loading these constants into local memory would consume too many resources and lead to program failure due to their large data volume. Therefore, we store these constants in the constant memory, enabling each thread to access them during computation, preventing program crashes.

IV. Performance Evaluation

Based on our optimized GPU-based vec implementation outlined in Section III, this section focuses on discussing the experimental results of implementing different parameter sets of Classic McEliece on the GPU platform and comparing the performance with other work.

The hardware environment for the CPU is a 16-core Intel(R) Core(TM) i5-12600KF v12 @ 3.70GHz. The hardware environment for the GPU is an NVIDIA GeForce RTX4090. The RTX4090 utilizes the Ada Lovelace architecture, featuring 16,384 CUDA cores and a clock frequency of 2.2 GHz. It also includes 512 tensor cores, 176 ROPs, and 128 RT cores. The memory capacity is 24 GB (GDDR6X), and the maximum bandwidth is 1,008 GB/s. The software environment includes Linux Ubuntu 20.04.4 LTS operating system and NVCC compiler. The compilation parameters are -O3 -std=c++11 -maxrregcount=128, with CUDA version 12.1.

We evaluate GPU performance from two perspectives:

- *Throughput*: The number of Classic McEliece encapsulation/decapsulation operations completed by the GPU per unit of time.
- *Latency*: Represents the time required for Classic McEliece encapsulation/decapsulation operations on the GPU platform, from the computation request to the completion of the computation.

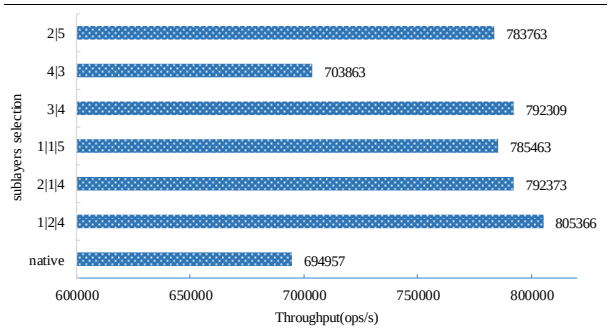
Additionally, during the experiments on the GPU platform, certain parameters affect algorithm performance. The specific explanations are as follows:

- *Block Number*: The number of blocks contained in a CUDA kernel.
- *Threads/Block*: The number of threads contained in a CUDA block.
- *Threads*: The total number of threads launched by the GPU kernel, calculated as the number of thread blocks multiplied by the number of threads per block.

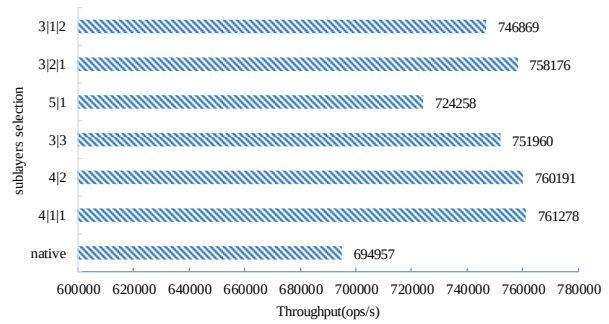
A. Layer Fusion of (T)AFFT

As discussed in Section III-C2 and III-D, we present two different layer fusion schemes for the TAFFT and AFFT, respectively. Here, we use experimental performance data to explain our choice of these particular layer fusion schemes.

Taking McEliece8192128 as an example, we utilize the GPU-based native decapsulation implementation as a baseline to evaluate the performance of different layer fusion schemes. Fig. 9(a) presents the decapsulation performance when applying different LFMASS strategies specifically for the transpose additive FFT. The native performance is 694,957 ops/s. The "1|2|4" scheme achieves the best performance, reaching 805,366 ops/s, resulting in a 15.89% performance improvement using this strategy. This is due to the strong correlation between the final parameter in the layer fusion scheme and the number of terminal nodes in the TAFFT stepping chain. A larger value for this parameter indicates fewer terminal nodes, while a smaller parameter indicates a higher number of terminal nodes. The greater the number of terminal nodes, the more complex the implementation, leading to a negative impact on decapsulation performance.



(a) Layer Fusion for TAFFT



(b) Layer Fusion for AFFT

Fig. 9. Performance Evaluation of Different Layer Fusion Schemes for McEliece8192128.

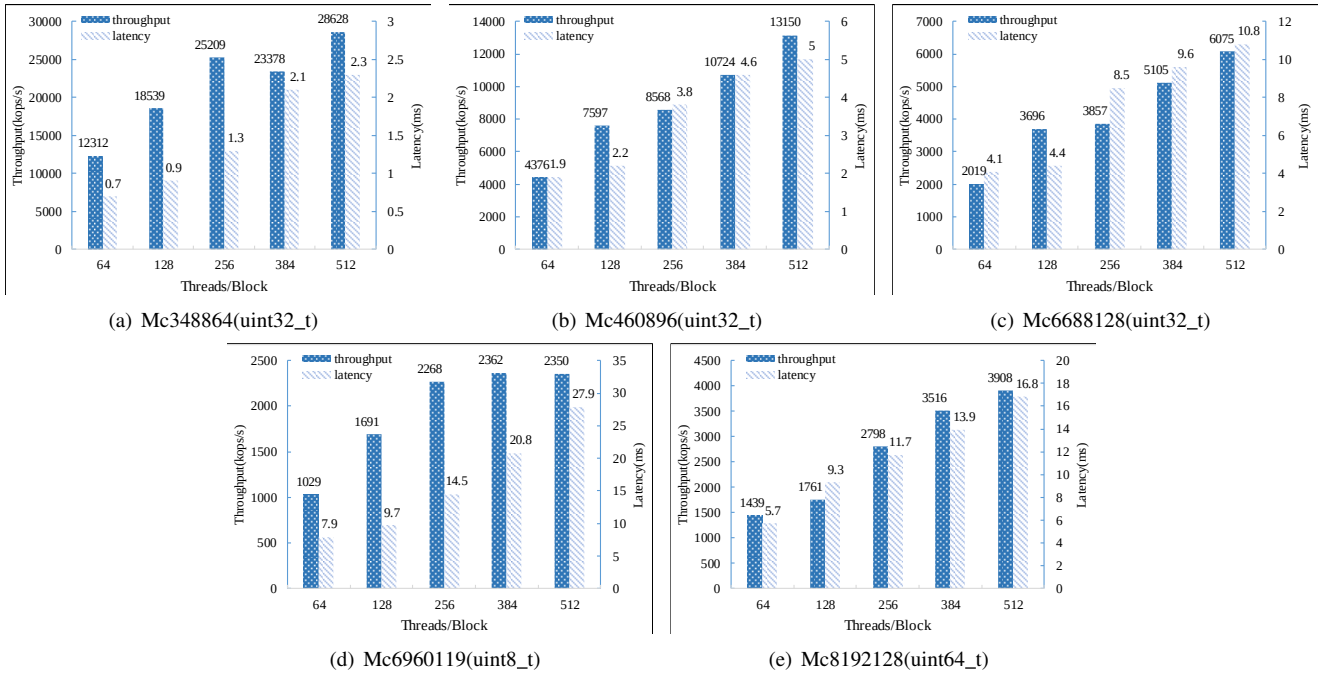
Conversely, with fewer terminal nodes, our LFMASS becomes more similar to the native strategy, thereby failing to effectively reduce address offset computations at each layer and not achieving significant performance improvements. Furthermore, for McEliece348864, the computation layers of the transpose additive FFT butterfly operation differ from other parameter sets. It only has six layers. Through experimentation, we ultimately selected the "1|1|4" layer fusion scheme for it.

Figure 9(b) illustrates the performance of the Classic McEliece decapsulation algorithm employing additive FFT with different layer fusion schemes. The "4|1|1" scheme demonstrates the best performance, achieving an approximate 9.5% improvement. Here, the first parameter of the layer fusion scheme is closely related to the number of terminal nodes in the AFFT stepping chain. The first parameter corresponds to more terminal nodes and greater complexity of the implementation, while a higher value aligns more closely with the native implementation approach.

B. Performance of Encapsulation

Given that the NVIDIA RTX4090 platform contains 128 multiprocessors, we set the number of blocks to 128 in order to fully utilize the GPU's computational resources. Within each block, the number of threads is set to 64, 128, 256, 384, and 512, respectively. Therefore, the total number of threads is 128×64, 128×128, 128×256, and 128×512, respectively.

As described in Section III-B, we implement encapsulation using the $uint32\_t$ data type, except for McEliece6960119, which employs $uint8\_t$ due to alignment constraints. In contrast, McEliece8192128 supports both $uint32\_t$ and $uint64\_t$



(a) Mc348864(uint32_t) (b) Mc460896(uint32_t) (c) Mc6688128(uint32_t)

(d) Mc6960119(uint8_t) (e) Mc8192128(uint64_t)

Fig. 10. Encapsulation Performance of Classic McEliece on the NVIDIA RTX4090.

implementations. Given that data types affect computational performance, we conducted a comprehensive evaluation, as presented in Table IV. The results show that the $uint32\_t$-based implementation performs slightly worse than its $uint64\_t$ counterpart. Although 32-bit operations are generally faster on GPUs, the encapsulation process using $uint32\_t$ requires nearly twice the number of operations compared to $uint64\_t$. Taking both execution speed and computational workload into account, the $uint64\_t$-based implementation demonstrates superior performance on the RTX4090. Therefore, we adopt the $uint64\_t$-based approach for McEliece8192128.

TABLE IV
ENCAPSULATION PERFORMANCE OF DIFFERENT IMPLEMENTATIONS FOR MCELIECE8192128

Threads	uint32_t		uint64_t	
	Throughput (ops/s)	Latency (ms)	Throughput (ops/s)	Latency (ms)
128×64	1,297,810	6.312	1,438,910	5.693
128×128	1,674,615	9.541	1,760,969	9.304
128×256	2,493,978	13.139	2,798,388	11.711
128×384	3,257,407	15.089	3,516,458	13.978
128×512	3,856,177	16.995	3,908,129	16.769

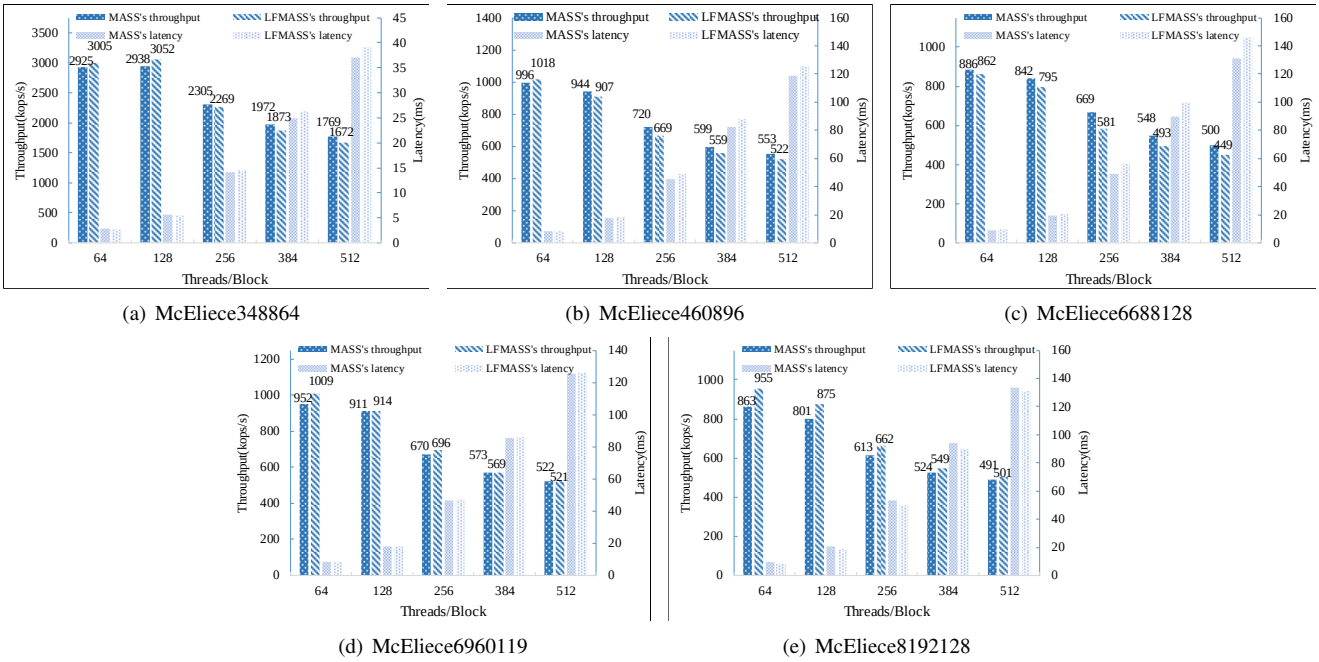
Figure 10 illustrates the performance of different Classic McEliece parameter sets for the encapsulation algorithm implemented on the RTX4090. The bracketed annotations in the subtitles of each subplot indicate the data types used in the encapsulation computation. The differences between the implementations of various parameter sets are explained in Section III-B. The encapsulation peak throughput for McEliece348864 and McEliece460896 can reach tens of millions of operations per second, while the remaining parameter sets achieve encapsulation throughput in millions of operations per second. As seen from the figure, with the increase in the number of encapsulation requests, the throughput performance consistently improves. For Classic McEliece encapsulation implementations with various parameter sets, the peak performance corresponds to a request number of approximately 128×512. Consequently, the latency also increases.

Theoretically, as the size of the public key increases, the computational requirements for executing the encapsulation algorithm increase as well. This results in longer encapsulation times and reduced performance. From the figure, it is evident that the encapsulation performance achieved with different parameter sets declines as the length of the Classic McEliece public key increases. Specifically, the performance in Fig. 10(d) is lower than in Fig. 10(e) because the encapsulation implementation of McEliece6960119 uses the $uint8\_t$ data type. Consequently, the computational steps involved in the encapsulation process are four times more than those using the $uint32\_t$ data type, leading to lower performance.

C. Performance of Decapsulation

In Section III, we introduce the concept of the TAFFT stepping chain and provide a detailed explanation of the MASS and LFMASS. We implement the McEliece decapsulation algorithm with different parameter sets on the RTX4090, and the performance results are shown in Fig. 11. For the parameter set with the shortest private key, McEliece348864, the decapsulation performance reaches up to 3,005 kops/s. For the other parameter sets, the performance can also reach approximately the millions of operations per second level.

It is observed that for McEliece348864, when the number of decapsulation requests is 128×128, the peak performance achieved by the two implementation strategies reaches



(a) McEliece348864 (b) McEliece460896 (c) McEliece6688128

(d) McEliece6960119 (e) McEliece8192128

Fig. 11. Decapsulation Performance of Classic McEliece on the NVIDIA RTX4090.

TABLE V
PEAK PERFORMANCE OF DIFFERENT IMPLEMENTATIONS OF DECAPSULATION

Parameter set	Native	MASS		LFMASS	
	Throughput(ops/s)	Throughput(ops/s)	Improvement(%)	Throughput(ops/s)	Improvement(%)
McEliece348864	2,353,705	2,937,989	24.82%	3,051,701	29.66%
McEliece460896	748,491	995,655	33.02%	1,018,249	36.04%
McEliece688128	683,431	886,257	29.68%	861,902	26.11%
McEliece6960119	728,958	951,734	30.56%	1,008,632	38.37%
McEliece8192128	694,957	863,491	24.25%	954,619	37.36%

2,925kops/s and 3,005kops/s, respectively. For all other parameter sets, the peak performance of decapsulation using these two implementation strategies is achieved at a request number of 128×64. After reaching the peak performance, the throughput performance declines as the number of decapsulation requests increases. This decline occurs because exceeding the peak performance causes a shortage of register resources, leading to data spillage into memory. This spillage results in decreased throughput performance and significantly increased latency. When the number of threads per block reaches 512, the latency can increase by more than an order of magnitude.

Table V illustrates the peak performance of different implementation methods for the McEliece decapsulation algorithm across various parameter sets. Initially, the baseline performance of the native GPU-based implementation, utilizing the native strategy for butterfly operations, is provided as a baseline. Additionally, the table encompasses the peak performance of the MASS and LFMASS, alongside their performance improvement ratios in comparison to the native implementation. The LFMASS achieves the highest performance improvement with McEliece6960119, at 38.37%. Overall, the decapsulation performance attained by LFMASS surpasses that of MASS, as corroborated by the throughput bar chart in Fig. 11. However, for McEliece688128, the MASS outperforms LFMASS.

D. Scalability Analysis

Our implementation is based on standard CUDA programming practices and does not rely on architecture-specific features, allowing for efficient portability across different generations of NVIDIA GPUs. To evaluate the generality and representativeness of our implementation across architectures, we ported the McEliece348864 CUDA code, without modifying the algorithmic logic or kernel design, to three additional NVIDIA platforms with varying compute capabilities: the Turing-based RTX 2080 Ti (CC 7.5), the Volta-based Titan V (CC 7.0), and the Ampere-based RTX 3090 (CC 8.6). All experiments were conducted under consistent software and compilation settings. The measured encapsulation and decapsulation throughput results are summarized in Table VI.

Experimental results demonstrate that the proposed GPU-based implementation of Classic McEliece can be compiled and executed efficiently on NVIDIA GPUs with different architectures, and its performance scales consistently with the available computational resources. As hardware capability increases, the algorithm's throughput rises accordingly, confirming the implementation's scalability and strong cross-

TABLE VI
PERFORMANCE OF MCELIECE348864 ACROSS MULTIPLE NVIDIA GPUS

Platform	Threads	Encap. (ops/s)	Decap.[1] (ops/s)	Decap.[2] (ops/s)
NVIDIA RTX 2080Ti	68×64	3,470,769	716,782	679,202
	68×128	5,990,873	789,680	780,814
	68×256	8,587,797	726,576	754,343
	68×384	9,570,564	694,054	714,261
	68×512	9,842,365	669,671	691,672
NVIDIA TiTan V	80×64	3,752,213	711,677	823,662
	80×128	6,090,652	852,459	862,468
	80×256	9,308,563	846,824	853,931
	80×384	11,160,022	801,286	817,887
	80×512	9,831,487	784,105	801,888
NVIDIA RTX 3090	82×64	4,052,078	1,130,211	1,235,152
	82×128	6,718,806	1,294,689	1,314,136
	82×256	10,693,391	1,216,647	1,293,056
	82×384	13,989,215	1,206,319	1,222,418
	82×512	16,757,986	1,197,502	1,201,318

[1] indicates that decapsulation is implemented using MASS.
[2] indicates that decapsulation is implemented using LFMASS.

device generalization. Given our design goal of maximizing throughput within acceptable latency, the results obtained on the RTX 4090 are representative, and the implementation can be ported to other GPU platforms with comparable efficiency, underscoring its broad applicability and practical value.

E. Comparison with Related Works

Recent research on GPU-based NTT optimization, such as [22] and [32], has demonstrated effective strategies for enhancing performance through architectural and memory access optimizations. The EDFS-NTT method in [22] proposes a depth-first memory access strategy, which shares some similarity with our MASS technique. However, due to the more complex data dependencies in TAFFT, MASS introduces tailored modifications to support efficient execution without incurring excessive synchronization overhead. Similarly, although the layer fusion strategy in [32] resembles our LFMASS in concept, our method is fundamentally different in that it adopts a coarse-grained parallel model where each thread computes a full TAFFT independently. This avoids inter-thread communication and control divergence, enabling better scalability and performance on modern GPUs.

Both [33] and [12] implement segments of Classic McEliece on the ARM Cortex-M4 @ 168MHz. The former primarily implements encapsulation for four parameter sets, whereas the latter, restricted by the flash size of the experimental platform, implements encapsulation for only two parameter sets with public keys smaller than 1MB. As indicated in Table VII, the encapsulation performance of the latter is approximately 5× higher compared to the former. Nevertheless, due to the computational constraints of the microprocessor, the performance remains relatively low. Similarly, the decapsulation throughput performance achieved on this platform is underwhelming, reaching only several tens of operations per second. [14] proposes an FPGA design for Classic McEliece that integrates encapsulation, decapsulation, and key generation modules. However, its implementation performance remains in the range of tens of thousands, whereas our throughput for the same parameter set is several thousand times greater.

Section I-A introduces the four official implementation versions of Classic McEliece. The official avx implementation provides the average number of clock cycles on an Intel Haswell CPU @ 3.10GHz. Based on this, latency and throughput are calculated [34], with specific data presented in Table VII. State-of-the-art CPU codes focus on minimising per-call latency within a largely sequential or lightly threaded execution model, hence their response time refers to a single encapsulation or decapsulation request. By contrast, our CUDA design launches hundreds to thousands of KEM instances in one kernel and therefore targets maximising aggregate throughput. The latency reported for the GPU is the batch-completion time, which also includes kernel-launch and synchronisation overhead, and is thus not directly comparable to the single-request latency measured on CPUs. Therefore, our focus is on analyzing throughput performance. Compared to the avx implementation, our GPU-based encapsulation performance has increased by 344× for the first three parameter sets and by over 100× for the last two parameter sets. Regarding decapsulation, compared to the avx implementation, our GPU-based decapsulation performance has improved by 125× for McEliece348864 and by approximately 90× for the other parameter sets. In deployment scenarios that involve large numbers of concurrent PQ-TLS or VPN, the bottleneck is the total number of KEM operations completed per unit time. Consequently, the GPU approach is well suited to high-concurrency, high-bandwidth environments, whereas CPU implementations remain preferable for endpoints where ultra-low single-operation latency is paramount.

The Table VII also provides several other key encapsulation mechanisms implemented on GPU platforms, such as Kyber, BIKE, and Frodo. The implementation of [22] shows significant performance improvements compared to [35]. Our implementation of the Classic McEliece encapsulation achieves 14× the performance of [22], despite their platform having superior performance to ours. However, the decryption performance of our Classic McEliece is weaker than that of Kyber, partly due to differences in the computational complexity of the algorithms themselves. Although BIKE is also a code-based post-quantum key encapsulation mechanism, the implementation by Dong et al. [37] on the NVIDIA RTX 4090 demonstrates significantly lower performance compared to our optimized implementation of Classic McEliece. The core performance bottleneck of BIKE lies in large-integer multiplication, which is fundamentally different from that of Classic McEliece. Therefore, a direct comparison in terms of optimization strategies is not applicable. Frodo is an LWE-based algorithm whose performance bottlenecks primarily stem from the computational overhead of AES operations and matrix multiplications. While CPU platforms, including those from Intel, benefit from dedicated hardware acceleration for AES, GPU platforms rely solely on software-based implementations and parallel optimization. This architectural limitation contributes to Frodo's inferior performance. Therefore, the GPU-based Frodo in [36] exhibits significantly lower performance compared to our implementation of Classic McEliece.

TABLE VII
PERFORMANCE COMPARISON OF CLASSIC MCELIECE

	Platform	Algorithm	Encapsulation			Decapsulation		
			Cycles	Latency (ms)	Throughput (ops/s)	Cycles	Latency (ms)	Throughput (ops/s)
[33]	ARM Cortex-M4 @ 168MHz	McEliece348864	3,106,183	18.489	54	-	-	-
		McEliece460896	5,868,529	34.932	29	-	-	-
		McEliece6688128	11,464,900	68.243	15	-	-	-
		McEliece8192128	14,696,239	87.478	11	-	-	-
[12]	ARM Cortex-M4 @ 168MHz	McEliece348864	582,199	3.465	289	2,706,681	16.111	62
		McEliece460896	1,081,335	6.437	155	6,535,186	38.900	26
		McEliece6688128	-	-	-	7,412,111	43.478	23
		McEliece8192128	-	-	-	7,481,747	45.455	22
[14]	Xilinx Artix 7 FPGA	McEliece348864	30,000	0.3	3,333	10,000	0.9	1,111
[34]	Intel Haswell CPU @ 3.10GHz	McEliece348864	37,585	0.012	82,480	127,668	0.041	24,282
		McEliece460896	81,312	0.026	38,125	263,634	0.085	11,759
		McEliece6688128	175,788	0.057	17,635	306,946	0.099	10,099
		McEliece6960119	147,192	0.047	21,061	287,218	0.093	10,793
		McEliece8192128	158,068	0.051	19,612	310,773	0.100	9,975
[35]	NVIDIA RTX3080	Kyber	-	-	1,298,701	-	-	2,380,952
[22]	NVIDIA Tesla V100	Kyber	-	-	2,105,977	-	-	9,378,937
[36]	NVIDIA RTX3090	Frodo	-	-	262,209	-	-	236,761
[37]	NVIDIA RTX4090	BIKE (1-thread)	-	519	236,319	-	16,256	7,558
		BIKE (3-thread)	-	147	277,789	-	7,040	5,817
Ours	NVIDIA RTX4090	McEliece348864	-	2.289	**28,628,195**	-	5.369	**3,051,701**
		McEliece460896	-	4.984	**13,149,787**	-	8.045	**1,018,249**
		McEliece6688128	-	10.788	**6,074,986**	-	9.243	**886,257**
		McEliece6960119	-	27.884	**2,350,301**	-	8.122	**1,008,632**
		McEliece8192128	-	16.769	**3,908,129**	-	8.581	**954,619**

V. CONCLUSION

We presented the first systematic implementation of the Classic McEliece KEM on NVIDIA GPU platforms, leveraging their powerful parallel computing capabilities. By utilizing a heterogeneous computing approach and optimizing memory access patterns, our implementation achieves encapsulation and decapsulation speeds of 28,628,195 ops/s and 3,051,701 ops/s for the McEliece348864, respectively. Our findings demonstrate significant performance improvements, with up to 344× faster encapsulation and 125× faster decapsulation compared to traditional CPU implementations. This work lays the groundwork for further enhancements in GPU-based post-quantum cryptography and underscores the potential of GPU platforms in securing future cryptographic applications. In future work, we will further investigate fine-grained parallelism strategies to achieve high-performance implementations of the Classic McEliece KEM.

REFERENCES

[1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM review*, vol. 41, no. 2, pp. 303–332, 1999.

[2] L. K. Grover, "Quantum computers can search rapidly by using almost any transformation," *Physical Review Letters*, vol. 80, no. 19, p. 4329, 1998.

[3] X. Zhou and X. Tang, "Research and implementation of rsa algorithm for encryption and decryption," in *Proceedings of 2011 6th international forum on strategic technology*, vol. 2, pp. 1118–1121, IEEE, 2011.

[4] D. Hankerson and A. Menezes, "Elliptic curve cryptography," in *Encyclopedia of Cryptography, Security and Privacy*, pp. 1–2, Springer, 2021.

[5] F. Opiłka, M. Niemiec, M. Gagliardi, and M. A. Kourtis, "Performance analysis of post-quantum cryptography algorithms for digital signature," *Applied Sciences*, vol. 14, no. 12, p. 4994, 2024.

[6] N. Aragon, P. Barreto, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, P. Gaborit, S. Ghosh, S. Gueron, T. Güneysu, *et al.*, "Bike: bit flipping key encapsulation," 2022.

[7] C. A. Melchor, N. Aragon, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, P. Gaborit, E. Persichetti, G. Zémor, and I. Bourges, "Hamming quasi-cyclic (hqc)," *NIST PQC Round*, vol. 2, no. 4, p. 13, 2018.

[8] T. Chou, C. Cid, S. UiB, J. Gilcher, T. Lange, V. Maram, R. Misoczki, R. Niederhagen, K. Paterson, and E. Persichetti, "Classic mceliece: conservative code-based cryptography, 10 october 2020," 2020.

[9] R. J. McEliece, "A public-key cryptosystem based on algebraic," *Coding Thv*, vol. 4244, pp. 114–116, 1978.

[10] T. C. Martin R. Albrecht, Daniel J. Bernstein, "Classic mceliece. technical report, national institute of stan dards and technology," 2024.

[11] N. Vettas, "Analysis and implementation of the post-quantum cryptographic scheme: Classic mceliece,"

[12] M.-S. Chen and T. Chou, "Classic mceliece on the arm cortex-m4," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 125–148, 2021.

[13] M. Sim, H. Kwon, S. Eum, G. Song, M. Lee, and H. Seo, "Efficient implementation of the classic mceliece on armv8 processors," in *International Conference on Information Security Applications*, pp. 324–337, Springer, 2023.

[14] P.-J. Chen, T. Chou, S. Deshpande, N. Lahr, R. Niederhagen, J. Szefer, and W. Wang, "Complete and improved fpga implementation of classic mceliece," *Cryptology ePrint Archive*, 2022.

[15] S. Chen, H. Lin, W. Huang, and Y. Huang, "Hardware design and implementation of classic mceliece post-quantum cryptosystem based on fpga," in *2022 IEEE High Performance Extreme Computing Conference (HPEC)*, pp. 1–6, 2022.

[16] D. J. Bernstein, T. Chou, and P. Schwabe, "Mcbits: fast constant-time code-based cryptography," in *Cryptographic Hardware and Embedded Systems-CHES 2013: 15th International Workshop, Santa Barbara, CA, USA, August 20-23, 2013. Proceedings 15*, pp. 250–272, Springer, 2013.

[17] H. Niederreiter, "Knapsack-type cryptosystems and algebraic coding theory," *Prob. Contr. Inform. Theory*, vol. 15, no. 2, pp. 157–166, 1986.

[18] T. Chou, "Mcbits revisited," in *International Conference on Cryptographic Hardware and Embedded Systems*, pp. 213–231, Springer, 2017.

[19] S. Gao and T. Mateer, "Additive fast fourier transforms over finite fields," *IEEE Transactions on Information Theory*, vol. 56, no. 12, pp. 6265–6272, 2010.

[20] V. E. Beneš, *Mathematical theory of connecting networks and telephone traffic*. Academic press, 1965.

[21] L. Gao, F. Zheng, N. Emmart, J. Dong, J. Lin, and C. Weems, "Dpf-ecc: Accelerating elliptic curve cryptography with floating-point computing power of gpus," in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 494–504, 2020.

[22] X. Ji, J. Dong, T. Deng, P. Zhang, J. Hua, and F. Xiao, "Hi-kyber: A novel high-performance implementation scheme of kyber based on gpu," *IEEE Transactions on Parallel and Distributed Systems*, 2024.

[23] S. Narisada, K. Fukushima, and S. Kiyomoto, "Fast gpu implementation of dumer's algorithm solving the syndrome decoding problem," in *2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom)*, pp. 971–977, IEEE, 2021.

[24] S. Narisada, S. Uemura, H. Okada, H. Furue, Y. Aikawa, and K. Fukushima, "Revisiting the may–meurer–thomae algorithm—solving mceliece-1409 in one day," *Cryptology ePrint Archive*, 2024.

[25] D. A. Rockenbach, C. M. Stein, D. Griebler, G. Mencagli, M. Torquati, M. Danelutto, and L. G. Fernandes, "stream processing on multi-cores with gpus: parallel programming models' challenges," in *2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pp. 834–841, IEEE, 2019.

[26] A. Habermaier and A. Knapp, "On the correctness of the simt execution model of gpus," in *European Symposium on Programming*, pp. 316–335, Springer, 2012.

[27] J. Sanders, *CUDA by Example: An Introduction to General-Purpose GPU Programming*. Addison-Wesley Professional, 2010.

[28] Y. Wang, "Revised quantum resistant public key encryption scheme rlce and ind-cca2 security for mceliece schemes.," *IACR Cryptol. ePrint Arch.*, vol. 2017, p. 206, 2017.

[29] W. Wang, J. Szefer, and R. Niederhagen, "Fpga-based niederreiter cryptosystem using binary goppa codes," in *Post-Quantum Cryptography: 9th International Conference, PQCrypto 2018, Fort Lauderdale, FL, USA, April 9-11, 2018, Proceedings 9*, pp. 77–98, Springer, 2018.

[30] S. Heyse and T. Güneysu, "Code-based cryptography on reconfigurable hardware: tweaking niederreiter encryption for performance," *Journal of Cryptographic Engineering*, vol. 3, pp. 29–43, 2013.

[31] NVIDIA, "NVIDIA GeForce RTX 4090." https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/, 2023.

[32] W.-K. Lee and S. O. Hwang, "High throughput implementation of post-quantum key encapsulation and decapsulation on gpu for internet of things applications," *IEEE Transactions on Services Computing*, vol. 15, no. 6, pp. 3275–3288, 2021.

[33] J. Roth, E. Karatsiolis, and J. Krämer, "Classic mceliece implementation with low memory footprint," in *Smart Card Research and Advanced Applications: 19th International Conference, CARDIS 2020, Virtual Event, November 18–19, 2020, Revised Selected Papers 19*, pp. 34–49, Springer, 2021.

[34] C. McEliece, "Cycle Counts for the October 2022 software on an Intel Haswell CPU core." https://classic.mceliece.org/impl.html, 2022.

[35] L. Wan, F. Zheng, G. Fan, R. Wei, L. Gao, Y. Wang, J. Lin, and J. Dong, "A novel high-performance implementation of crystals-kyber with ai accelerator," in *European Symposium on Research in Computer Security*, pp. 514–534, Springer, 2022.

[36] W.-K. Lee, H. Seo, S. O. Hwang, R. Achar, A. Karmakar, and J. M. B. Mera, "Dpcrypto: Acceleration of post-quantum cryptography using dot-product instructions on gpus," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 9, pp. 3591–3604, 2022.

[37] J. Dong, Y. Fu, X. Qin, Z. Dong, F. Xiao, and J. Lin, "Eco-bike: Bridging the gap between pqc bike and gpu acceleration," *IEEE Transactions on Information Forensics and Security*, 2024.